TALLER DOM

Fundamentos del DOM -Sofia Alejandra Trujillo Ordoñez

9 DE OCTUBRE DE 2026

Parte 1. Fundamentos del DOM

¿Qué significa DOM y qué palabras componen esta sigla?

DOM significa **Document Object Model** (Modelo de Objetos del Documento).

¿Qué es el DOM y para qué sirve en el desarrollo web?

El **DOM** es una interfaz de programación de aplicaciones (API) para documentos HTML y XML. Define la estructura lógica de los documentos y la forma en que se accede y manipula un documento. Sirve para que los lenguajes de programación (como JavaScript) puedan modificar dinámicamente la estructura, el estilo y el contenido de una página web en tiempo de ejecución.

¿Cómo representa el navegador un documento HTML mediante el DOM?

El navegador convierte el código HTML fuente en un **árbol de nodos jerárquico**. Cada etiqueta HTML, atributo y bloque de texto se convierte en un objeto manipulable en memoria con propiedades y métodos.

¿Qué relación existe entre HTML, CSS, JavaScript y el DOM?

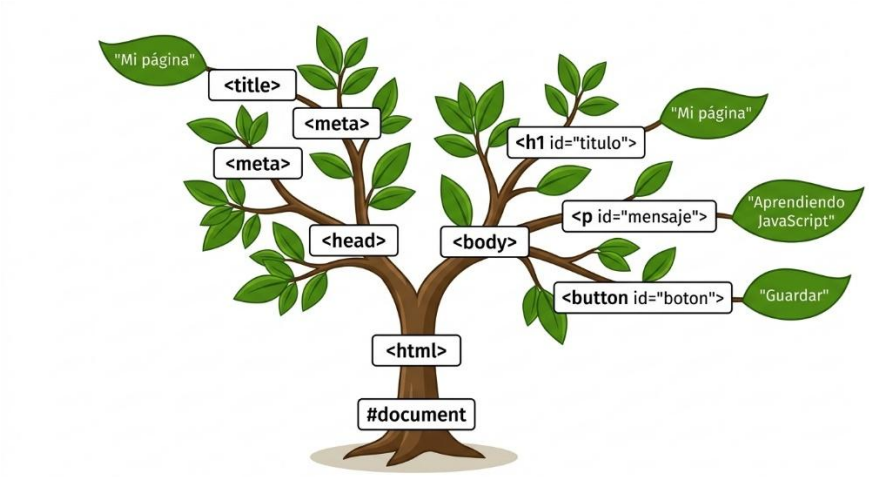
HTML: Proporciona la estructura inicial y estática del documento.
CSS: Define los estilos visuales de los elementos del documento.
DOM: Es la representación estructurada en memoria del HTML y CSS generada por el navegador.
JavaScript: Es el lenguaje que interactúa con el DOM para consultar, crear, modificar o eliminar elementos dinámicamente en respuesta a eventos del usuario.

¿Qué es un nodo en el DOM? Explique la diferencia entre un nodo de elemento, un nodo de texto y un nodo de documento.

Un **nodo** es el punto básico de construcción dentro del árbol del DOM.

- **Nodo de documento (DocumentNode):** Representa todo el documento web (document). Es la raíz del árbol.
- **Nodo de elemento (ElementNode):** Representa cada etiqueta HTML (ej. <h1>, <p>, <button>).
- **Nodo de texto (TextNode):** Representa el texto contenido dentro de un elemento HTML (por ejemplo, el texto "Mi página" dentro de <h1>Mi página</h1>).

¿Qué es el árbol DOM? Dibuje un árbol que represente la estructura HTML siguiente:



Esta metáfora visual demuestra que las etiquetas HTML no son simples líneas de texto, sino una estructura viva de nodos conectados. JavaScript no modifica la hoja de código fuente directamente, sino que viaja por las ramas de este árbol para alterar los textos (hojas/TextNodes), cambiar estilos o reaccionar a los eventos del usuario.

¿Qué ocurriría con una página web si JavaScript no pudiera acceder al DOM?

La página web sería puramente estática. No se podrían realizar validaciones de formularios en tiempo real, cargar contenido dinámico (AJAX/Fetch), crear animaciones complejas, alternar temas visuales (modo oscuro/claro) ni responder a interacciones del usuario sin recargar toda la página desde el servidor.

¿El DOM es lo mismo que el código HTML escrito por el desarrollador? Justifique su respuesta con un ejemplo.

No. El código HTML escrito es el archivo fuente original en texto plano. El DOM es el modelo en memoria dinámico construido por el navegador.

- *Ejemplo:* Si en HTML escribimos y luego con JavaScript ejecutamos document.getElementById('caja').textContent = 'Hola', el código fuente HTML original sigue estando vacío, pero el DOM tiene ahora un nodo de texto dinámico con la palabra "Hola".

Parte 2. Selección y manipulación de elementos

Investigue cada método o propiedad. Explique su utilidad, indique qué devuelve o modifica y construya un ejemplo propio.

Método o propiedad	Descripción y utilidad	¿Qué devuelve o modifica?	Ejemplo propio
getElementById()	Selecciona un elemento mediante su ID único en el documento.	Devuelve una referencia al único objeto Element encontrado o null.	const titulo = document.getElementById("main-title");
getElementsByClassName()	Selecciona todos los elementos con una clase CSS determinada.	Devuelve una colección HTML viva (HTMLCollection).	const items = document.getElementsByClassName("item");
getElementsByTagName()	Selecciona elementos según el nombre de la etiqueta HTML.	Devuelve un HTMLCollection con todos los elementos de esa etiqueta.	const parrafos = document.getElementsByTagName("p");
querySelector()	Selecciona el **primer** elemento que coincida con el selector CSS dado.	Devuelve el primer Element que coincide o null si no hay coincidencias.	const primerBoton = document.querySelector(".btn-active");
querySelectorAll()	Selecciona **todos** los elementos que coinciden con un selector CSS.	Devuelve un NodeList estático con los elementos encontrados.	const tarjetas = document.querySelectorAll(".card");
innerHTML	Obtiene o cambia el marcado HTML completo dentro del elemento.	Modifica/Devuelve un string interpretando etiquetas HTML.	caja.innerHTML = "Texto en negrita";

textContent	Obtiene o establece únicamente el texto plano de un elemento y sus descendientes.	Modifica/Devuelve un string sin interpretar HTML.	parrafo.textContent = "Texto plano sin marcas.";
style	Permite acceder y modificar propiedades CSS en línea del elemento.	Modifica estilos inline (element.style.propiedad).	caja.style.backgroundColor = "green";
classList	Manipula las clases CSS asociadas al elemento (add, remove, toggle, contains).	Permite añadir, quitar, alternar o verificar clases.	caja.classList.toggle("destacado");
createElement()	Crea un nuevo elemento HTML especificado por su nombre de etiqueta.	Devuelve un nuevo nodo Element.	const nuevoDiv = document.createElement("div");
appendChild()	Añade un nodo al final de la lista de hijos de un nodo padre especificado.	Añade el nodo y devuelve el nodo insertado.	contenedor.appendChild(nuevoDiv);
remove()	Elimina el elemento directamente de la estructura del DOM.	No devuelve nada (undefined).	elementoEliminar.remove();

Actividad de análisis: Diferencias entre innerHTML y textContent

- **innerHTML:** Interpreta la cadena asignada como marcado HTML, parseando las etiquetas e insertándolas como nuevos nodos.
- **textContent:** Trata la cadena estrictamente como texto plano, escapando cualquier etiqueta HTML para evitar que el navegador la ejecute.

ejemplo_ejercicio.html > html

```html

<!DOCTYPE html>
<html>
<body>

<h2>Comparación de métodos</h2>

<p id="resultado1"></p>
<p id="resultado2"></p>

<script>
let texto = "<b>Hola, estoy aprendiendo JavaScript</b>";

document.getElementById("resultado1").innerHTML = texto;
document.getElementById("resultado2").textContent = texto;
</script>

</body>
</html>
```

Comparación de métodos

Hola, estoy aprendiendo JavaScript

Hola, estoy aprendiendo JavaScript

innerHTML permite insertar contenido HTML e interpretar sus etiquetas. En cambio, textContent solo inserta texto y no convierte las etiquetas en elementos HTML. Por eso, en este ejemplo el primer resultado aparece en negrita y el segundo muestra las etiquetas escritas. Cuando solo necesitamos mostrar texto, textContent suele ser una opción más segura, especialmente si el contenido proviene de un usuario.

Parte 3. Investigación sobre eventos en JavaScript

Conceptos fundamentales

¿Qué es un evento en JavaScript?

Es una señal o notificación de que algo ha sucedido en la página web (una interacción del usuario o un cambio del sistema/navegador).

¿Por qué los eventos son importantes para crear páginas web interactivas?

Permite que las páginas respondan activamente a las acciones del usuario.

¿Qué relación existe entre un evento, un elemento HTML y una función de JavaScript?

Un **evento** ocurre en un **elemento HTML**, el cual es detectado para desencadenar una **función JavaScript** (manejador).

¿Qué significa escuchar o detectar un evento?

Significa tener un script pendiente a que se produzca un evento específico sobre un nodo específico.

¿Qué es un manejador de eventos (event handler)?

Es la función ejecutada en respuesta a la activación de un evento.

¿Qué diferencia existe entre un evento y una función?

El evento es la *acción/suceso* (ej. click), mientras que la función es el *bloque de código* que se ejecuta como consecuencia.

¿Qué es addEventListener() y cuál es su sintaxis general?

Método recomendado para adjuntar un escuchador de eventos sin sobrescribir otros.

```
elemento.addEventListener("tipoEvento", funcionCallback, usoCaptura);
```

¿Qué diferencia existe entre utilizar onclick directamente en HTML y utilizar addEventListener() desde JavaScript?

onclick en HTML mezcla la lógica JS con la estructura HTML y solo permite un evento por elemento; addEventListener() separa las tecnologías y permite registrar múltiples funciones al mismo evento.

¿Qué sucede si se registra más de un manejador para el mismo tipo de evento y elemento?

Con addEventListener() todos los manejadores agregados se ejecutarán secuencialmente en el orden registrado.

¿Qué es el objeto event y qué información puede proporcionar cuando ocurre una interacción?

Es un objeto automático que se pasa como argumento a la función manejadora. Contiene información contextual de la interacción (coordenadas del mouse, tecla presionada, elemento que originó el evento, etc.).

Evento	Cuándo ocurre	Elemento ideal	Caso de uso real
click	Al presionar y soltar el botón primario del mouse sobre un elemento.	<button>, <a>	Confirmar una compra o enviar un pedido.
dblclick	Al hacer doble clic rápido en un mismo elemento.	, <img>	Ampliar una imagen o editar una fila de tabla.
mouseover	Al mover el puntero del mouse sobre un elemento o sus hijos.	,	Desplegar un menú emergente o submenu.
mouseout	Al mover el puntero fuera de un elemento o de uno de sus hijos.	,	Ocultar un menú cuando el usuario retira el cursor.
mouseenter	Al mover el puntero dentro de un elemento (no se propaga a hijos).	<section>, .card	Aplicar un efecto de iluminación en una tarjeta.
mouseleave	Al salir el cursor del área del elemento (no se propaga).	<section>, .card	Quitar la animación visual de la tarjeta.
keydown	Al presionar cualquier tecla en el teclado.	<input>, window	Mover un personaje con las flechas en un juego web.
keyup	Al soltar la tecla previamente presionada.	<input>	Validar si el campo cumple los caracteres mínimos.
input	Síncronamente cuando el valor de un campo cambia.	<input>, <textarea>	Contador de caracteres restantes en un campo de texto.
change	Al cambiar el valor y perder el foco (o seleccionar opción).	<select>, <input type="checkbox">	Cambiar la lista de ciudades al elegir un país.
submit	Al presionar el botón de envío o dar Enter dentro del formulario.	<form>	Validar datos antes de enviarlos a la API backend.

focus	Cuando un elemento recibe la atención del usuario.	<input>, <button>	Resaltar el borde del campo con un color especial.
blur	Cuando un elemento pierde la atención/foco.	<input>	Verificar si el email ingresado tiene formato correcto.
load	Cuando la página o un recurso termina de cargarse completamente.	window, <img>	Quitar la pantalla de carga (*loader*) del sitio web.

¿Por qué un botón puede utilizar click, mientras que un campo de texto puede utilizar input?

Uso de click vs input: Un botón es un elemento de activación puntual (click), mientras que un campo de texto requiere registrar los cambios progresivos de contenido (input).

¿Qué diferencia existe entre mouseenter y mouseover?

mouseenter vs mouseover: mouseenter no implementa burbujeo (*bubbling*); no se dispara al pasar sobre elementos hijos. mouseover sí burbujea.

¿Qué diferencia existe entre keydown y keyup?

keydown vs keyup: keydown ocurre al pulsar hacia abajo (se repite si se mantiene presionada la tecla); keyup ocurre una sola vez al soltar la tecla.

¿Por qué se utiliza preventDefault() al trabajar con determinados formularios?

preventDefault() en formularios: Evita la acción predeterminada del navegador (como recargar la página al enviar un formulario), permitiendo validar los datos con JavaScript antes de hacer un envío asíncrono.

¿Qué sucede cuando se ejecuta event.target? Construya un ejemplo.

event.target: Hace referencia al elemento exacto sobre el cual ocurrió la acción originaria del evento.

```
document.addEventListener("click", (e) => console.log(e.target));
```

¿Qué diferencia existe entre event.target y event.currentTarget?

event.target vs event.currentTarget: event.target es el elemento originario que detonó el evento; event.currentTarget es el elemento al que está adjuntado el escuchador de eventos.

¿Qué son la propagación de eventos, el bubbling y la captura?

- **Captura (*Capturing*):** El evento viaja desde la raíz del DOM hasta el elemento objetivo.
- **Burbujeo (*Bubbling*):** El evento viaja desde el elemento objetivo ascendiendo hacia la raíz.

¿En qué consiste la delegación de eventos y cuándo podría ser útil?

Delegación de eventos: Técnica que consiste en agregar un único escuchador de eventos a un elemento padre en lugar de adjuntarlo a múltiples hijos (aprovechando el burbujeo). Es muy útil cuando se agregan o eliminan elementos de forma dinámica.

Parte 4. Taller práctico de investigación

Ejercicio 1. Botón interactivo

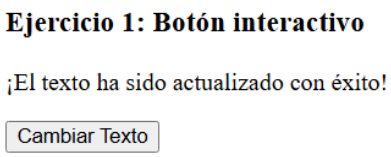
Cree una página con un botón y un párrafo. Al hacer clic en el botón, debe cambiar el texto del párrafo. Investigue: ¿cómo seleccionó el párrafo?, ¿qué evento utilizó?, ¿qué propiedad modificó?

```html
<section class="seccion-caja">
  <h3>Ejercicio 1: Botón interactivo</h3>
  <p id="parrafo1">Texto inicial del párrafo.</p>
  <button id="btnCambiarTexto">Cambiar Texto</button>
</section>
```

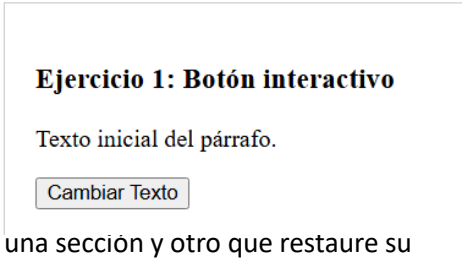
```javascript
const p1 = document.getElementById("parrafo1");
const btn1 = document.getElementById("btnCambiarTexto");
btn1.addEventListener("click", () => {
  p1.textContent = "¡El texto ha sido actualizado con éxito!";
});
```

Seleccionado mediante getElementById(), evento click, modificando la propiedad textContent.

En un inicio



Al dar click



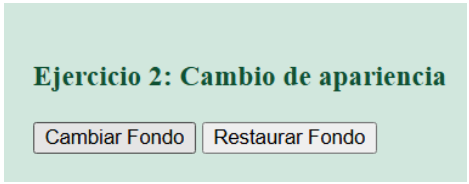
Cree un botón que cambie el color de fondo de una sección y otro que restaure su apariencia original. Investigue: ¿es mejor modificar style o utilizar clases CSS con classList? Justifique.

```html
<section id="seccionColor" class="seccion-caja">
  <h3>Ejercicio 2: Cambio de apariencia</h3>
  <button id="btnColorApp">Cambiar Fondo</button>
  <button id="btnRestaurarApp">Restaurar Fondo</button>
</section>
```

```javascript
const secColor = document.getElementById("seccionColor");
document.getElementById("btnColorApp").addEventListener("click", () => secColor.classList.add("fondo-activo"));
document.getElementById("btnRestaurarApp").addEventListener("click", () => secColor.classList.remove("fondo-activo"));
```

Es preferible utilizar clases CSS con classList, ya que mantiene la separación entre la capa de presentación (CSS) y la lógica (JavaScript), facilitando la mantenibilidad.





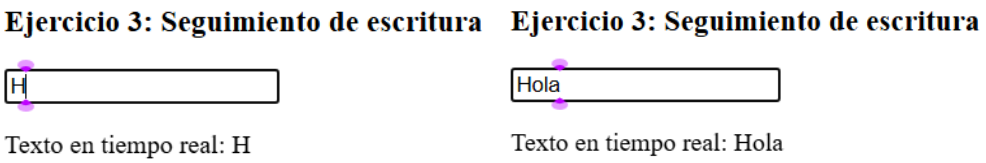
Ejercicio 3. Seguimiento de escritura

Cree un campo de texto y un párrafo que muestre en tiempo real lo que escribe el usuario. Investigue: ¿por qué es apropiado utilizar input en vez de click? ¿Qué información se obtiene de la propiedad value?

```html
<section class="seccion-caja">
  <h3>Ejercicio 3: Seguimiento de escritura</h3>
  <input type="text" id="campoEscritura" placeholder="Escribe algo aquí ...">
  <p>Texto en tiempo real: <span id="vistaPrevia"></span></p>
</section>
```

```javascript
const campo = document.getElementById("campoEscritura");
const vista = document.getElementById("vistaPrevia");
campo.addEventListener("input", (e) => {
  vista.textContent = e.target.value;
});
```

input captura cambios en tiempo real en cada pulsación/pegado; click requeriría presionar fuera o en un botón. value retorna el contenido actual introducido por el usuario.



Ejercicio 4. Lista dinámica

Cree un campo de texto y un botón para agregar elementos a una lista. Cada elemento debe poder eliminarse individualmente. Investigue: ¿para qué sirven createElement(), appendChild() y remove()? ¿Cómo se asigna el evento al botón de eliminar?

```html
<section class="seccion-caja">
  <h3>Ejercicio 4: Lista dinámica</h3>
  <input type="text" id="nuevoItemText" placeholder="Nuevo elemento">
  <button id="btnAgregarItem">Agregar a la Lista</button>
  <ul id="listaDinamica"></ul>
</section>
```

```
const inputLista = document.getElementById("nuevoItemText");
const btnLista = document.getElementById("btnAgregarItem");
const ulLista = document.getElementById("listaDinamica");

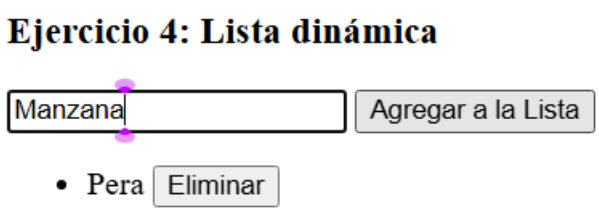
btnLista.addEventListener("click", () => {
  if (!inputLista.value.trim()) return;

  const li = document.createElement("li");
  li.textContent = inputLista.value + " ";

  const btnEliminar = document.createElement("button");
  btnEliminar.textContent = "Eliminar";
  btnEliminar.addEventListener("click", () => li.remove());

  li.appendChild(btnEliminar);
  ulLista.appendChild(li);
  inputLista.value = "";
});
```

createElement() crea el nodo, appendChild() lo inserta en el árbol y remove() elimina el elemento. El evento de eliminar se asigna dinámicamente al crear cada botón.



Ejercicio 5. Formulario con validación

Construya un formulario con nombre, correo electrónico y contraseña. Al enviarlo, valide que los campos estén diligenciados y muestre mensajes de error cuando corresponda. Investigue: ¿cómo funciona submit?, ¿para qué sirve preventDefault()?, ¿cuál es la diferencia entre los atributos HTML de validación y la validación con JavaScript?

```html
<section class="seccion-caja">
  <h3>Ejercicio 5: Formulario con validación</h3>
  <form id="formRegistro">
    <div>
      <label>Nombre:</label>
      <input type="text" id="nombreInput">
      <span id="errNombre" class="error"></span>
    </div>
    <div>
      <label>Correo:</label>
      <input type="text" id="correoInput">
      <span id="errCorreo" class="error"></span>
    </div>
    <div>
      <label>Contraseña:</label>
      <input type="password" id="passInput">
      <span id="errPass" class="error"></span>
    </div>
    <button type="submit">Enviar</button>
  </form>
</section>
```

```javascript
const form = document.getElementById("formRegistro");
form.addEventListener("submit", (e) => {
  e.preventDefault();

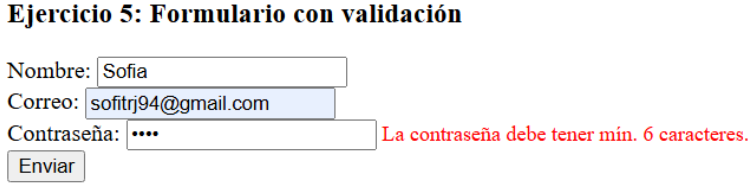
  let valido = true;
  const nombre = document.getElementById("nombreInput").value.trim();
  const correo = document.getElementById("correoInput").value.trim();
  const pass = document.getElementById("passInput").value.trim();

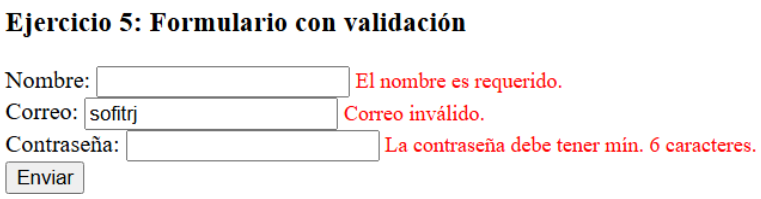
  document.getElementById("errNombre").textContent = nombre ? "" : " El nombre es requerido.";
  document.getElementById("errCorreo").textContent = correo.includes("@") ? "" : " Correo inválido.";
  document.getElementById("errPass").textContent = pass.length >= 6 ? "" : " La contraseña debe tener mín. 6 caracteres.";

  if (!nombre || !correo.includes("@") || pass.length < 6) valido = false;

  if (valido) alert("Formulario enviado correctamente.");
});
```

submit intercepta el intento de envío. preventDefault() cancela el refresco automático de la página. La validación HTML es nativa del navegador pero menos personalizable; la de JavaScript permite lógica avanzada de negocio y mensajes personalizados.





¿Qué elementos se seleccionan mediante getElementById()?

Se seleccionan tres elementos del DOM utilizando sus identificadores únicos (id):

- **document.getElementById("titulo"):** Selecciona el elemento de encabezado principal <h1 id="titulo">.
- **document.getElementById("mensaje"): Selecciona** el elemento de párrafo <p id="mensaje">.
- **document.getElementById("boton"):** Selecciona el elemento de botón interactivo <button id="boton">.

¿Qué función cumple addEventListener() en este código?

Actúa como un **escuchador de eventos** adjuntado al botón. Su función es estar pendiente de la interacción del usuario y, cuando se detecta un **clic (click)** sobre el botón, ejecuta automáticamente la función (*callback*) asignada.

¿Qué sucedería si se reemplaza click por dblclick?

El evento ya no responderá con un clic sencillo. La función asignada solo se ejecutará cuando el usuario haga **doble clic rápido** sobre el botón. Si hace un único clic, la página no realizará ninguna acción.

¿Qué cambia cuando se ejecuta la función?

Al presionar el botón se ejecutan tres cambios inmediatos:

1. **Texto del párrafo:** El contenido de <p id="mensaje"> cambia a "El botón fue presionado".
2. **Color del título:** La propiedad CSS de color del <h1 id="titulo"> cambia a **azul (blue)**.
3. **Consola:** Se imprime en la consola del navegador la referencia HTML del elemento que recibió el clic.

¿Qué información muestra console.log(event.target)?

Muestra en la consola de desarrollador del navegador el **elemento HTML exacto que originó el evento**. En este caso, imprime la etiqueta completa del botón:

```
<button id="boton">Presionar</button>
```

¿Qué pasaría si el identificador del botón fuera incorrecto?

Si el id no coincide (por ejemplo, si se busca "botonn"), document.getElementById() devolverá null. Al intentar ejecutar boton.addEventListener(...), JavaScript arrojará un error no controlado en la consola:

```
TypeError: Cannot read properties of null (reading 'addEventListener')
```

Debido a esto, la ejecución del script se detendrá por completo en esa línea.

¿Qué errores podrían ocurrir si se intenta acceder a un elemento que no existe?

- **TypeError: Cannot read properties of null**: Ocurre al intentar leer o modificar propiedades/métodos (como .addEventListener, .textContent o .style) sobre una variable cuyo valor es null por no haber encontrado el elemento en el DOM.
- I**nterrupción del script**: La ejecución del código se bloquea a partir del error, evitando que las líneas posteriores de JavaScript se ejecuten.

Modifique el código para que el título cambie de color cada vez que se presione el botón, alternando entre azul y rojo.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
    <title>Investigación del DOM</title>
</head>
<body>

  <h1 id="titulo">Mi aplicación</h1>
  <p id="mensaje">Esperando una acción...</p>
  <button id="boton">Presionar</button>

    <script>
  const titulo = document.getElementById("titulo");
  const mensaje = document.getElementById("mensaje");
  const boton = document.getElementById("boton");

  boton.addEventListener("click", function(event) {
    mensaje.textContent = "El botón fue presionado";

    // Alterna el color entre azul y rojo en cada clic
    if (titulo.style.color === "blue") {
      titulo.style.color = "red";
    } else {
      titulo.style.color = "blue";
    }

    console.log(event.target);
  });

</script>

</body>
</html>
```



Explique cómo comprobaría que el evento funciona sin depender únicamente de la apariencia visual.

Existen varias alternativas técnicas para verificar el funcionamiento:

1. **Salida por consola (console.log)**: Imprimir un mensaje de confirmación dentro de la función (ej. console.log("Evento ejecutado con éxito");).
2. **Inspector de elementos del navegador (DevTools - F12)**: Observar en la pestaña *Elements* cómo se actualizan dinámicamente las propiedades del nodo en el árbol DOM cuando ocurre la acción.

3. **Uso de puntos de interrupción (*Breakpoints*)**: Pausar la ejecución del código en el panel de fuentes (*Sources*) para verificar que la ejecución entra en la función cuando se presiona el botón.

Proponga una mejora para separar el código JavaScript del documento HTML.

Para cumplir con las buenas prácticas de desarrollo (*Separación de Responsabilidades*):

1. **Crear un archivo independiente:** Mover la lógica de JavaScript a un archivo exclusivo (por ejemplo, script.js).

2. **Vincular el archivo en el HTML:** Quitar las etiquetas <script> internas y enlazar el archivo al final del <body> mediante el atributo src:

```
<script src="script.js"></script>
```

3. **Manejar estilos mediante clases CSS:** En lugar de modificar los estilos inline desde JS (titulo.style.color), es mejor agregar/quitar clases con classList.toggle() para mantener la capa de presentación dentro de una hoja de estilos .css.

Parte 6. Investigación aplicada a un caso real

Seleccione un sistema informático que utilice habitualmente, como un sistema de matrícula, una tienda virtual, una aplicación de citas o una plataforma educativa. Elabore un análisis que incluya:

Nombre y propósito del sistema.

Sistema seleccionado: Territorium / **SOFIA Plus** (Plataforma Educativa del SENA).
Propósito del sistema: Administrar el proceso de aprendizaje virtual y presencial, permitiendo a los aprendices e instructores acceder a contenidos de cursos, entregar evidencias de aprendizaje, responder evaluaciones, consultar calificaciones y recibir avisos informativos.

Tres elementos de la interfaz con los que interactúa el usuario.

• Elemento de la interfaz	• Evento de JavaScript	• Acción que ejecuta JavaScript cuando ocurre el evento
• **1. Campo de búsqueda de evidencias / cursos** (<input type="text">)	• **input** •	• Filtra dinámicamente en tiempo real la lista de actividades o guías mostradas en pantalla a medida que el aprendiz escribe cada letra, sin necesidad de recargar la página.
• **2. Botón para cargar / adjuntar archivo de evidencia** (<button>)	• **click** •	• Abre la ventana del explorador de archivos local y, al seleccionar la tarea, actualiza el estado de la vista mostrando el nombre del archivo cargado y activando el botón de envío.
• **3. Formulario de envío de evidencias** (<form>)	• **submit** •	• Valida que los campos requeridos (comentario y archivo adjunto) estén diligenciados; previene el envío directo con preventDefault(), envía los datos al servidor mediante una petición asíncrona (AJAX/Fetch) y muestra una alerta de confirmación.

La relación entre el usuario y la plataforma mediante el DOM ocurre en tres etapas fundamentales:

Construcción e Identificación:

Cuando la página de SOFIA Plus se carga, el navegador lee la estructura HTML y construye el **Árbol DOM**. JavaScript accede a los elementos HTML usando selectores del DOM (como document.getElementById o document.querySelector) y les conecta un "escuchador" (addEventListener).

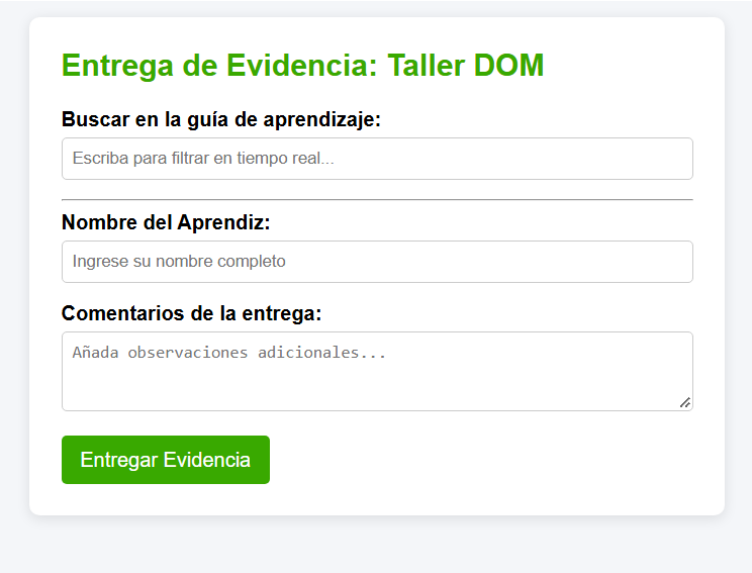
Detección del Evento:

Cuando el aprendiz interactúa con la pantalla (por ejemplo, al hacer clic en *Entregar Evidencia*), el navegador detecta la acción y genera un objeto de evento (event) que viaja por el árbol DOM.

Manipulación Dinámica del Árbol DOM:

En respuesta al evento, JavaScript no modifica el código fuente de la página, sino la representación en memoria del DOM:

- **createElement() y appendChild():** Crea nuevos nodos HTML (como un mensaje de confirmación verde en pantalla) y los inserta dentro de la estructura.
- **classList.toggle() / style:** Modifica los atributos o estilos visuales de nodos existentes para mostrar u ocultar paneles de notas.
- **textContent / innerHTML:** Actualiza el texto de los contadores (por ejemplo, cambiar el estado de *"Pendiente"* a *"Entregado"*).





Entrega de Evidencia: Taller DOM

Buscar en la guía de aprendizaje:

Taller DOM

Filtrando contenido por: Taller DOM

Nombre del Aprendiz:

Ingrese su nombre completo

Comentarios de la entrega:

Añada observaciones adicionales...

Entregar Evidencia

¡Evidencia entregada con éxito por Sofía Trujillo! Fecha: 18:54:40